AMERICAN LATIN CLASS ATTENDANCE SYSTEM

Python Analytics Api

Table of Contents

Python Analytics API

Purpose

The project now includes a Python API for the Advanced Web Development requirement. It is a FastAPI microservice focused on analytical academic views for the American Latin Class Attendance System.

The existing Node.js/Express API remains responsible for authentication, sessions, enrollment, CRUD workflows, attendance registration and reports. The Python API reads the same PostgreSQL/RDS database and exposes analytics that help evaluate attendance risk, scholarship readiness, branch performance and teacher workload.

Framework And Responsibility

Area	Decision
Language	Python

Area	Decision
Framework	FastAPI
Runtime server	Uvicorn
Database access	`psycopg` to PostgreSQL/RDS
Authentication	Shared JWT session token issued by the Node Auth API
Base path	`/api/analytics/v1`
AWS target port	`8000`

This service is intentionally separate from the required Node.js backend so it can be deployed as an additional API microservice while preserving the existing academic architecture.

Authentication Model

The Python API does not create users or sessions. Instead, it validates the existing project session token:

1. The tester logs in through the Node Auth API using `POST /api/v1/auth/login`.
2. The Node API returns `data.sessionToken`.
3. Postman stores that token in `{{session_token}}`.
4. Python analytics requests send `Authorization: Bearer {{session_token}}`.
5. The Python API validates the JWT signature with the same `SESSION_SECRET`.
6. The Python API hashes the token and verifies that the session exists in PostgreSQL, is not revoked and has not expired.
7. After `POST /api/v1/auth/logout`, the same token is rejected by both Node and Python APIs.

This proves backend-only authentication and session revocation in Postman without requiring the frontend.

Endpoints

Method	URI	Auth	W
`GET`	`/api/analytics/v1/health`	Public	C Py ru

Method	URI	Auth	W
`GET`	`/api/analytics/v1/students/{student_id}/attendance-risk`	Bearer JWT	C at pe le re fc
`GET`	`/api/analytics/v1/students/{student_id}/scholarship-readiness`	Bearer JWT	C st at ag ac sc
`GET`	`/api/analytics/v1/branches/{branch_id}/performance-summary`	Bearer JWT	Su st te se at pe en ou
`GET`	`/api/analytics/v1/teachers/{teacher_id}/workload-summary`	Bearer JWT	C te in ho es

Optional date filters for student and teacher analytics:

Query Parameter	Example	Applies To
`from`	`2026-06-01T00:00:00Z`	Student attendance risk, scholarship readiness, teacher workload
`to`	`2026-07-01T00:00:00Z`	Student attendance risk, scholarship readiness, teacher workload

Response Format

Successful response:

```
{
  "success": true,
  "message": "Student attendance risk",
```

```
  "data": {
    "studentId": "85f4bbe9-5d5f-4126-89b6-ddd9de432885",
    "attendanceRate": 90,
    "riskLevel": "low"
  }
}
```

Unauthorized response:

```
{
  "success": false,
  "message": "Authentication required",
  "data": null
}
```

Postman Verification

Import:

- `postman/American-Latin-Class.postman_environment.json`
- `postman/American-Latin-Class-API.postman_collection.json`
- `postman/American-Latin-Class-Analytics-API.postman_collection.json`

Recommended order:

1. Select the `American Latin Class - AWS` environment.
2. Run `Auth & Session / Current Session - No Token`; expected result: `401`.
3. Run `Auth & Session / Password Login - Demo`; expected result: `200` and `session_token` saved automatically.
4. Run `Python Analytics API / Student Attendance Risk - No Token`; expected result: `401`.
5. Run the remaining Python Analytics API requests with inherited Bearer Token authorization.
6. Run `Session Teardown / Logout`.
7. Run a Python analytics protected request again with the old token; expected result: `401`.

Environment Variables

```
DATABASE_URL=postgres://alc_user:<password>@american-latin-
class.c38uoym8e77j.us-east-2.rds.amazonaws.com:5432/american_latin_class
SESSION_SECRET=<same value used by the Node Auth API>
ANALYTICS_AUTH_REQUIRED=true
ANALYTICS_CORS_ORIGINS=https://18-217-255-109.sslip.io
ANALYTICS_SERVICE_NAME=American Latin Class Analytics API
```

Secrets must stay out of Git. Store them in the EC2 `.env` file or in AWS Systems Manager Parameter Store / Secrets Manager.

AWS Deployment Target

Recommended infrastructure:

AWS Resource	Value
Instance name	Python Analytics API EC2
Instance type	`t3.micro`
OS	Ubuntu
Port	`8000`
Process	`uvicorn app.main:app --host 0.0.0.0 --port 8000`
Public routing	Nginx proxy from Frontend EC2 path `/api/analytics/v1`
Database access	RDS inbound PostgreSQL `5432` from the Python API security group

Frontend Nginx can expose it as:

```
https://18-217-255-109.sslip.io/api/analytics/v1
```

and proxy internally to:

```
http://<python-analytics-private-ip>:8000/api/analytics/v1
```

Local Validation

```
cd 06Code/python-analytics-api
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python -m unittest discover -s tests -v
```

The tests validate service calculations and the health endpoint.